

使用 Alpha-Beta 方法設計一個黑白棋 AI

張簡雲翔¹

Othello AI Homework

110321018¹

1. Code Analysis

1.0 Code source

本次作業本身基於老師所提供的範例程式，完成可以與對弈平台互動的一個部分，然後在自己撰寫 AlphaBeta 的 code，以及修改 evaluate function 完成的。

1.1 OthelloGame

在下載下來的檔案中，有一個檔案是 OthelloGame.py，這份檔案主要是在記錄一些跟遊戲棋盤、遊戲結束與否判斷，以及把棋盤顯示出來的函式，簡而言之，就是跟遊戲介面或者說主要遊玩時的畫面有關的資訊。不過因為本次是要連上對弈平台來使用，所以這個檔案在正式比賽的時候沒有用上。但是我有跟我朋友要了一個可以讓兩隻機器人在電腦上對下的程式(test.py)，就是使用這個 code 來執行的。如果今天是要做強化式學習，才會需要用到這個來產生模擬環境。

1.2 OthelloUtil

跟 OthelloGame.py 同樣資料夾的檔案中，有另一個檔案叫做 OthelloUtil.py，這個檔案主要是在寫一些遊戲規則相關的檔案，像是哪邊有合法的可下位置，或者當下之後，有哪些棋子要顏色對調等，在判斷遊戲規則以確保遊戲符合黑白棋的規則。用於找哪些是合法步，以及在 alpha-beta 中再做展開與模擬棋盤的時候會用到。

2. Code Modify

2.1 AlphaBeta.py

在原始的 Sample code 中所提供的只有一支隨機亂下的機器人，所以我把原本的機器人的檔案複製出來，改名叫做 AlphaBeta.py。並把所有跟 AlphaBeta 相關的改動都寫在裡面，以方便後面的呼叫。簡而言之，這次主要的跟唯一的改動都是在這個檔案裡面。

2.2 alphabeta

這個是放在 AlphaBeta.py 中的一個函式，主要是參考我們作業三所寫過的 Alpha-Beta search 的過程，進行展開與剪枝的動作，展開的部分主要是使用到 OthelloUtil 中的函式，去模擬對手以及自己下的棋步，並結合後面的 evaluate，去計算現在這個下法的分數，並進行剪枝的動作。

2.3 evaluate

這個一樣是放在 AlphaBeta.py 中的一個函式，主要工作是提供給 alphabeta 當作 evaluate function 來使用的。也是這次的作業中比較多可以修改跟優化的地方。當時設計了兩版不同的 evaluation function，在 source code 中所放的是第二版，因為第二版是第一版的一個延伸，以下進行仔細的說明：

第一版：

概念十分的粗暴與簡單，就是去計算模擬出來的盤面中，有多少自己的棋跟對手的棋，並進行相減，得到 score。

第二版：

因為第一版的想法實在太簡單，偶爾還會下輸給隨機亂下的機器人，所以上網找了一下黑白棋的規則，以及參考一下其他人主要設計那些東西作為 evaluate function。主要分成四個部分：

1. 模擬出來的盤面中，對手跟自己的棋差多少子。(與第一版同)
2. 在黑白棋規則中，角落十分的重要，所以計算雙方所佔據的角落數量

3. 在棋盤中，有些位置很容易被別人翻過去，有些位置不大會被別人翻過去，網路上找到一個權重表，我把它改成 6*6 可以用的版本，權重如下：

4	-3	2	2	-3	4
-3	-4	-1	-1	-4	-3
2	-1	1	1	-1	2
2	-1	1	1	-1	2
-3	-4	-1	-1	-4	-3
4	-3	2	2	-3	4

然後去把現在棋盤上，如果有子的去計算他所佔的位置的權重做加總。

4. 計算每一步有多少可以移動的步數

在有上面四個部分後，再做一個再把四項進行權重加總，得到最後的 score，經過調整與思考權重如下：

$$\text{Score} = 10 * 1. + 10 * 2. + 5 * 3. + 4.$$

至於這樣的權重設計的想法，主要是因為我認為最後決定勝敗的是關於雙方各至持有的子數，所以給比較重的權重。然後在黑白棋中，角落基本上就是不會被翻，甚至有時候角落可能會成為最終翻盤的決定性關鍵，所以也給比較重的權重。然後對於全局而言，棋子會不會被翻，相對而言也跟角落有沒有子有關，所以也會加重角落的權重。然後全局相較於到底有多少可動步數比較重要，但不如前面兩項重要，所以給予第二高的權重。最後最不重要的可移動步數就給予最低的權重。然後以五為倍數，如果問我原因就是我很喜歡五這個數字。就得到上面的公式。

3. Conclusion

在最後成功有成功從初賽晉級，但是在決賽晉級一次後，就被刷掉了，有些人他們調整的 evaluate function，以及模擬的步數，真的是很難匹敵。因為我之前有先修過機器學習，在上次主要是訓練很花時間，在這次是在想要怎麼設計 evaluate function 很花時間，是一個十分不同的體驗。使用機器學方法有時候程式設計師可以不懂黑白棋規則，但是使用這種比較傳統的方法，就會很考驗程式設計師對於規則的熟悉度。這次的比賽與作業，讓我對於人工智

慧方法有更深刻的認識以及學習，但是我還是要說：「那個弄到十三層的，他是不是真的太誇張了。讓不讓別人有參賽體驗拉，真的是，不要讓他打初賽拉」，咳咳咳咳，沒事。